Question 2 — Database Testing answer script

Part A — Schema & Structural Validation

1.

```
SHOW TABLES;
DESCRIBE users;
DESCRIBE usage_logs;
```

2.

```
-- Show all Primary Keys
SELECT
   TABLE_NAME,
   COLUMN_NAME,
   CONSTRAINT_NAME
FROM
   INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE
   TABLE_SCHEMA = 'ai_saas_db'
   AND CONSTRAINT_NAME = 'PRIMARY'
ORDER BY TABLE_NAME;

-- Show all Foreign Keys
SELECT
   TABLE_NAME,
   COLUMN_NAME,
   CONSTRAINT_NAME,
   REFERENCED_TABLE_NAME,
   REFERENCED_COLUMN_NAME
FROM
   INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE
   TABLE_SCHEMA = 'ai_saas_db'
   AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, CONSTRAINT_NAME;
```

3.

```
-- Test 1: Email Uniqueness (second insert should FAIL)
INSERT INTO users (email, full_name, plan, status)
VALUES ('test@example.com', 'Test User', 'free', 'active');

INSERT INTO users (email, full_name, plan, status)
```

```
VALUES ('test@example.com', 'Duplicate User', 'pro', 'active');
-- Expected: Error 1062

-- Test 2: Invalid Plan (should FAIL)
INSERT INTO users (email, full_name, plan, status)
VALUES ('invalidplan@example.com', 'Invalid User', 'platinum', 'active');
-- Expected: Error 1265

-- Clean up
DELETE FROM users WHERE email IN ('test@example.com', 'invalidplan@example.com');
```

Part B — CRUD & Constraint Testing

1.
```
INSERT INTO users (email, full_name, plan, status)
VALUES ('newuser@example.com', 'New User', 'free', 'active');

SELECT * FROM users WHERE email = 'newuser@example.com';
```

2.
```
INSERT INTO users (email, full_name, plan, status)
VALUES ('newuser@example.com', 'Duplicate User', 'pro', 'active');
-- Expected: Error 1062
```

3.
```
INSERT INTO users (email, full_name, plan, status)
VALUES ('invalidplan@example.com', 'Invalid User', 'platinum', 'active');
-- Expected: Error 1265
```

4.
```
-- Before update
SELECT user_id, email, full_name, plan
FROM users
WHERE email = 'newuser@example.com';

-- Update
UPDATE users
SET plan = 'pro'
WHERE email = 'newuser@example.com';

-- After update
```

```
SELECT user_id, email, full_name, plan
FROM users
WHERE email = 'newuser@example.com';
```

5.

```
-- Step 1: Create test data
INSERT INTO users (email, full_name, plan, status)
VALUES ('deletetest@example.com', 'Delete Test', 'free', 'active');

SET @user_id = LAST_INSERT_ID();

INSERT INTO chat_sessions (user_id, title)
VALUES (@user_id, 'Test Session');

SET @session_id = LAST_INSERT_ID();

INSERT INTO messages (session_id, role, content)
VALUES
   (@session_id, 'user', 'Test message 1'),
   (@session_id, 'assistant', 'Test response 1');

INSERT INTO api_keys (user_id, key_value, name, status)
VALUES (@user_id, 'delete_test_key', 'Test Key', 'active');

-- Step 2: Verify BEFORE
SELECT 'BEFORE:' as Status;
SELECT * FROM users WHERE user_id = @user_id;
SELECT * FROM chat_sessions WHERE user_id = @user_id;
SELECT * FROM messages WHERE session_id = @session_id;
SELECT * FROM api_keys WHERE user_id = @user_id;

-- Step 3: Delete (CASCADE)
DELETE FROM users WHERE user_id = @user_id;

-- Step 4: Verify AFTER
SELECT 'AFTER:' as Status;
SELECT * FROM users WHERE user_id = @user_id;
SELECT * FROM chat_sessions WHERE user_id = @user_id;
SELECT * FROM messages WHERE session_id = @session_id;
SELECT * FROM api_keys WHERE user_id = @user_id;
```

Part C — Data Integrity & Business-Rule Queries

1.
```
SELECT
   u.user_id,
   u.email,
   u.full_name,
   u.plan,
   u.status,
   u.created_at
FROM users u
LEFT JOIN subscriptions s ON u.user_id = s.user_id
WHERE s.subscription_id IS NULL;
```

2.
```
SELECT
   u.user_id,
   u.email,
   u.full_name,
   u.plan,
   COALESCE(SUM(l.tokens_used), 0) AS total_tokens_used,
   COALESCE(SUM(l.cost), 0) AS total_cost,
   COUNT(l.log_id) AS total_requests
FROM users u
LEFT JOIN usage_logs l ON u.user_id = l.user_id
GROUP BY u.user_id, u.email, u.full_name, u.plan
ORDER BY total_cost DESC;
```

3.
```
SELECT
   cs.session_id,
   u.full_name AS owner_name,
   u.email AS owner_email,
   COUNT(m.message_id) AS message_count,
   cs.created_at AS session_started,
   cs.updated_at AS session_updated
FROM chat_sessions cs
JOIN users u ON cs.user_id = u.user_id
JOIN messages m ON cs.session_id = m.session_id
GROUP BY cs.session_id, u.full_name, u.email, cs.created_at, cs.updated_at
ORDER BY message_count DESC
LIMIT 1;
```

4.

```
SELECT
    ak.api_key_id,
    ak.key_value,
    ak.name AS key_name,
    ak.status,
    ak.created_at,
    ak.last_used,
    u.email AS owner_email,
    u.full_name AS owner_name
FROM api_keys ak
JOIN users u ON ak.user_id = u.user_id
WHERE ak.status = 'revoked';
```

Part D — Transactions & Reliability (5 marks)

1.

```
-- Show BEFORE state
SELECT 'BEFORE COMMIT:' as Status;
SELECT * FROM users WHERE email = 'bob@example.com';
SELECT * FROM subscriptions WHERE user_id = (SELECT user_id FROM users WHERE
email = 'bob@example.com');

-- Start transaction
START TRANSACTION;

-- Update user's plan from 'pro' to 'enterprise'
UPDATE users
SET plan = 'enterprise'
WHERE email = 'bob@example.com';

-- Insert matching subscription
INSERT INTO subscriptions (user_id, plan, start_date, status)
SELECT
    user_id,
    'enterprise',
    CURDATE(),
    'active'
FROM users
WHERE email = 'bob@example.com';

-- Verify changes within transaction
SELECT 'DURING TRANSACTION:' as Status;
```

```
SELECT * FROM users WHERE email = 'bob@example.com';
SELECT * FROM subscriptions WHERE user_id = (SELECT user_id FROM users WHERE
email = 'bob@example.com')
ORDER BY subscription_id DESC LIMIT 1;

-- Commit all changes
COMMIT;

-- Show AFTER state (changes persisted)
SELECT 'AFTER COMMIT:' as Status;
SELECT * FROM users WHERE email = 'bob@example.com';
SELECT * FROM subscriptions WHERE user_id = (SELECT user_id FROM users WHERE
email = 'bob@example.com')
ORDER BY subscription_id DESC LIMIT 1;
```

2.

```
-- Show BEFORE state
SELECT 'BEFORE ROLLBACK:' as Status;
SELECT * FROM users WHERE email = 'bob@example.com';
SELECT * FROM subscriptions WHERE user_id = (SELECT user_id FROM users WHERE
email = 'bob@example.com')
ORDER BY subscription_id DESC LIMIT 1;

-- Start transaction
START TRANSACTION;

-- Attempt to update user's plan
UPDATE users
SET plan = 'free'
WHERE email = 'bob@example.com';

-- Attempt to insert subscription
INSERT INTO subscriptions (user_id, plan, start_date, status)
SELECT
    user_id,
    'free',
    CURDATE(),
    'active'
FROM users
WHERE email = 'bob@example.com';

-- Verify changes within transaction (temporarily visible)
```

```
SELECT 'DURING TRANSACTION (will be rolled back):' as Status;
SELECT * FROM users WHERE email = 'bob@example.com';
SELECT * FROM subscriptions WHERE user_id = (SELECT user_id FROM users WHERE
email = 'bob@example.com')
ORDER BY subscription_id DESC LIMIT 1;

-- Rollback all changes
ROLLBACK;

-- Show AFTER state (should be unchanged from BEFORE)
SELECT 'AFTER ROLLBACK (unchanged):' as Status;
SELECT * FROM users WHERE email = 'bob@example.com';
SELECT * FROM subscriptions WHERE user_id = (SELECT user_id FROM users WHERE
email = 'bob@example.com')
ORDER BY subscription_id DESC LIMIT 1;
```